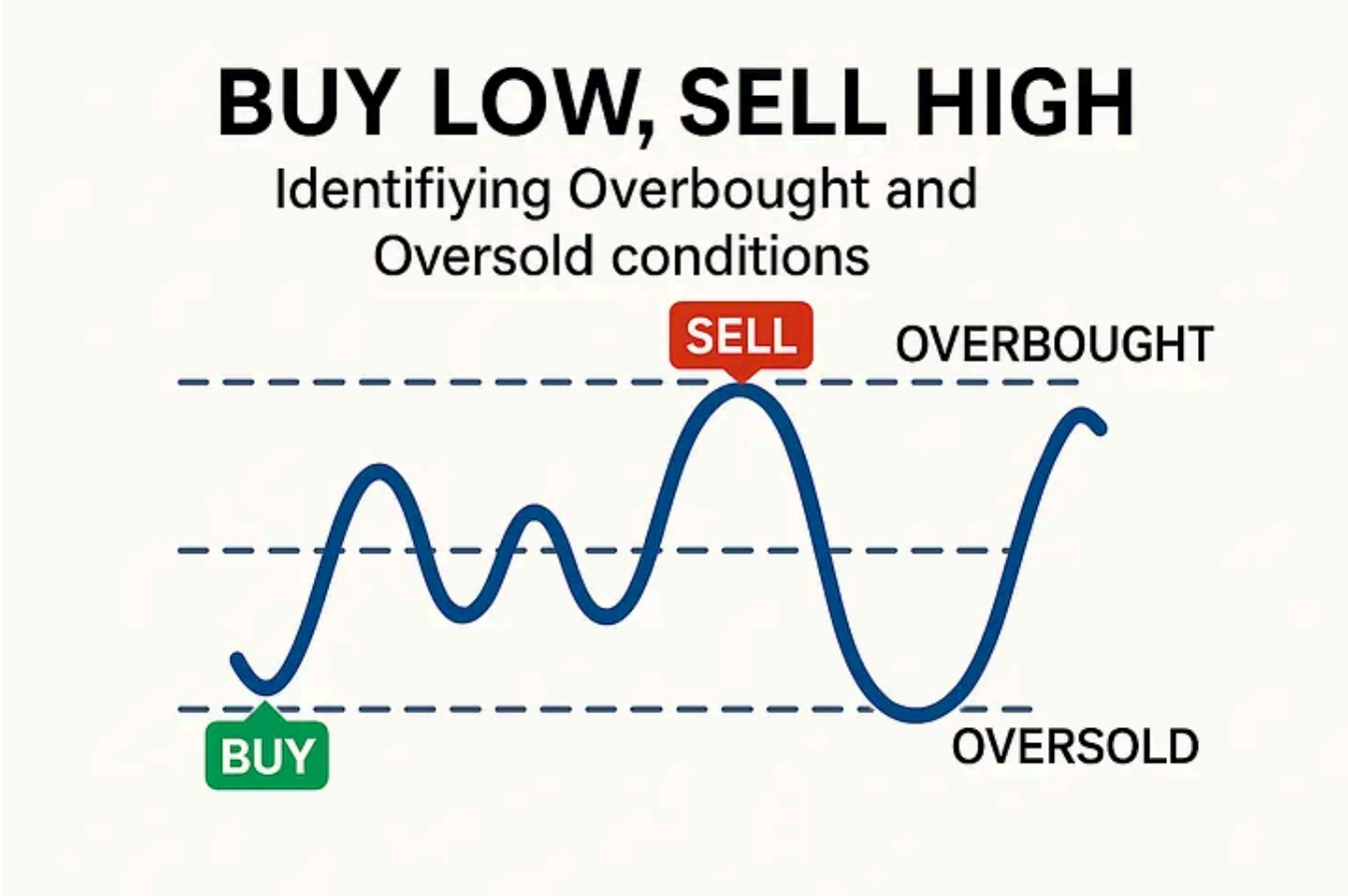
Stock Market Regimes Multi-class Classification

12 min read · Mar 10, 2026

Simon Leung

Identify and Predict five Market Regimes using 1D-CNN with Technical Indicators



Generated by Copilot

Overbought conditions occur when an asset's price has risen sharply and rapidly, often pushing it above its intrinsic value. This typically reflects excessive buying pressure, suggesting the price may be unsustainably high and vulnerable to a

pullback — creating a potential opportunity to "sell high."
Conversely, ***oversold*** conditions arise when the price has fallen quickly, potentially dropping below intrinsic value due to heavy selling pressure. This can signal that the asset is undervalued and may be poised for a rebound, presenting a potential "buy low" opportunity.
Together, these conditions often indicate that price momentum may be nearing exhaustion and that a reversal could be approaching, making them valuable signals for timing long entries in oversold markets and short entries in overbought markets.

I've read two interesting articles where Convolutional Neural Networks (CNNs) are used for stock-market regime prediction:

1. S&P 500 Stock's Movement Prediction using CNN by Rahul Gupta, 2025 [code from github]
2. STOCK BUY-SELL-HOLD PREDICTION USING CNN by Quant Club, IIT Kharagpur, Mar 3, 2024 [code from github]. (which similar to Stock Buy/Sell Prediction Using Convolutional Neural Network
   By Asutosh Nayak, Jan 19, 2020)

Below are the key summary highlights.

	S&P 500 Stocks Movement Prediction using CNN	**Stock Buy-Sell-Hold Prediction using CNN**
Prediction Target	Binary classification: BUY/SELL (bullish/bearish)	Ternary classification: BUY/SELL/HOLD
Input Features	Raw market data: OHLCV + Adjusted OHLCV (10 channels total)	Engineered technical indicators (116 features, reduced to 81) + OHLCV
Labeling Strategy	Predict BUY/SELL based on the absolute return.	11-day window: If middle value is maximum → sell; if minimum → buy; else hold.
Input Format	1D time series treated as multi-channel data [batch_size, window_size, 10 channels]	2D images created from tabular data (reshaped to 5×4 or 9×9 images)
Model Architecture	10-layer 1D-CNN	shallower 2D-CNN
Performance Metrics	Accuracy up to 91% (JPM 30-day forecasting)	93% accuracy after addressing class imbalance
Technical Indicators Used	None (raw price data only)	SMA, RSI, WILLR, and 100+ other indicators

Let me explore an approach that uses six bounded technical indicators to classify five market conditions — ['STRONG_OVERBOUGHT', 'OVERBOUGHT', 'NEUTRAL',

‘OVERSOLD’, ‘STRONG_OVERSOLD’] — and then incorporates an additional five unbounded technical indicators as features for a 1D-CNN model to predict these five stock-market regimes.

Here is my steps [Full code is here]:
1. Fetch 10 years S&P500 index (^SPX) stock prices from Yahoo finance.
2. Use 6 bounded technical indicators (using Python ta library) to consensus label market regimes:
[‘STRONG_OVERBOUGHT’, ‘OVERBOUGHT’, ‘NEUTRAL’, ‘OVERSOLD’, ‘STRONG_OVERSOLD’]
3. Add additional 5 unbounded technical indicators (using Python ta library) as Feature Engineering
4. Do Random Forest feature selection for all columns includes OHLCV
5. Dealing with unbalanced classes using SMOTE + class weight
6. Create 12-layers of Convolutional Neural Network Architecture using PyTorch with Early stopping + LR scheduler
7. Scaling, Training, Model Evaluation
8. Create comprehensive visualization, Classification report + Confusion Matrix, Training & Validation loss and accuracy curves, Learning rate curve

. . .

Step #1

```
# Fetch S&P500 historical data
ticker = "^SPX"
start_date = '2016-01-01'
end_date = '2026-01-01'  # datetime.now()
#start_date = end_date - timedelta(days=365*10)  # 10 year of data

data = yf.download(ticker, start=start_date, end=end_date, progress=False)

# Flatten the multi-level columns to a single level, yfinance returns MultiIndex
data.columns = data.columns.get_level_values(0)
data.columns = [''.join(col).strip() for col in data.columns.values]
data = data.reset_index()

# Set Date as DatetimeIndex
data['Date'] = pd.to_datetime(data['Date'])
data = data.set_index('Date')
```

```
print(f"Downloaded {len(data)} days of ^SPX data")
print(f"Date range: {data.index[0].strftime('%Y-%m-%d')} to {data.index[-1].strftim
data.head()
```

```
Downloaded 2514 days of ^SPX data
Date range: 2016-01-04 to 2025-12-31
```

Step #2

Indicator	Range	STRONG_OVERBOUGHT	OVERBOUGHT	NEUTRAL	OVERSOLD	STRONG_OVERSOLD
RSI (Relative Strength Index)	0-100	>80	70-80	30-70	20-30	<20
Stochastic Oscillator (%K)	0-100	>90	80-90	20-80	10-20	<10
Stochastic Oscillator (%D)	0-100	>90	80-90	20-80	10-20	<10
Stochastic RSI	0-100	>0.90	0.80-0.90	0.20-0.80	0.10-0.20	<0.10
Williams %R (Williams Percent Range)	-100 to 0	<-90	-80 to -90	-80 to -20	-20 to -10	>-10
MFI (Money Flow Index)	0-100	>90	80-90	20-80	10-20	<10
Ultimate Oscillator (7, 14, 28)	0-100	>80	70-80	30-70	20-30	<20

Image from [1]

Image from [2]

Image from [3]

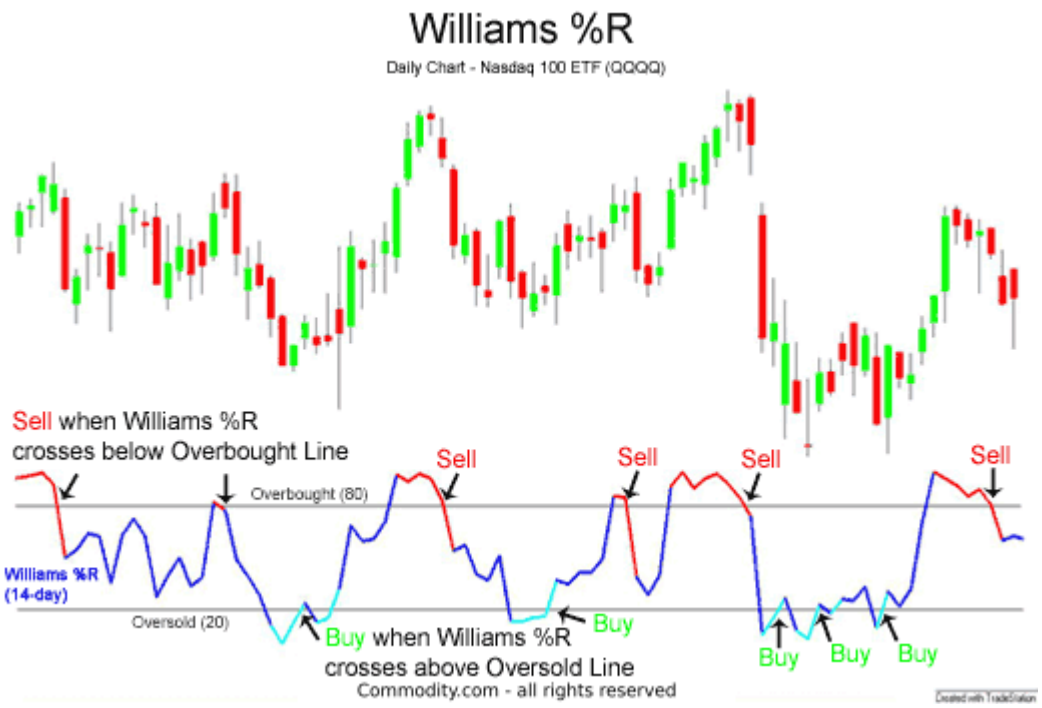


Image from [4]

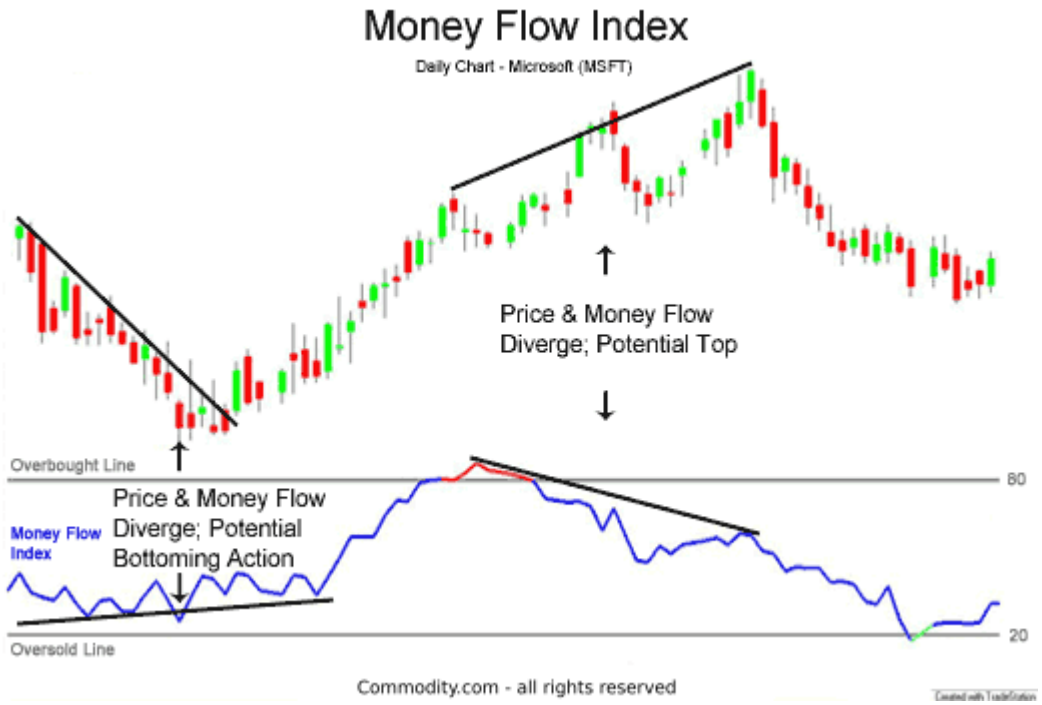


Image from [5]

Image from [6]

```
# ------------------------------------
#  Compute technical indicators (Using ta library)
# ------------------------------------
# Bounded indicators:
#   - RSI (14 period)
#   - Stochastic Oscillator Fast (%K, %D)
#   - Williams %R (14 period)
#   - Stochastic RSI
#   - Money Flow Index (MFI) (14 period)
#   - Ultimate Oscillator (7, 14, 28 periods)


def calculate_bounded_indicators(df):

    # 1. RSI (14 period)
    rsi = RSIIndicator(close=df['Close'], window=14)
    df['RSI'] = rsi.rsi()

    # 2. Stochastic Fast (%K and %D)
    stoch_fast = StochasticOscillator(
        high=df['High'],
        low=df['Low'],
        close=df['Close'],
        window=14,
        smooth_window=3
    )
    df['STOCH_K'] = stoch_fast.stoch()
    df['STOCH_D'] = stoch_fast.stoch_signal()

    # 3. Stochastic Slow
```

```
        df['STOCH_SLOW_K'] = df['STOCH_D']
        df['STOCH_SLOW_D'] = df['STOCH_SLOW_K'].rolling(window=3).mean()

        # 4. Stochastic RSI
        stoch_rsi = StochRSIIndicator(close=df['Close'], window=14, smooth1=3, smooth2=
        df['STOCHRSI'] = stoch_rsi.stochrsi() * 100

        # 5. Williams %R
        high_14 = df['High'].rolling(window=14).max()
        low_14 = df['Low'].rolling(window=14).min()
        df['WILLIAMS_R'] = ((high_14 - df['Close']) / (high_14 - low_14)) * -100

        # 6. MFI (14 period)
        mfi = MFIIndicator(
            high=df['High'],
            low=df['Low'],
            close=df['Close'],
            volume=df['Volume'],
            window=14
        )
        df['MFI'] = mfi.money_flow_index()

        # 7. Ultimate Oscillator
        ultimate = UltimateOscillator(
            high=df['High'],
            low=df['Low'],
            close=df['Close'],
            window1=7,
            window2=14,
            window3=28
        )
        df['ULTIMATE_OSC'] = ultimate.ultimate_oscillator()

        df.dropna(inplace=True)
        return df
```

Identify 5 statuses logic:

1. **Count & Weight**: Tally how many indicators are in each category. Give "Strong" signals **2 points**, regular signals **1 point**, neutral **0 points**.
2. **Check for Conflict**: If both Overbought AND Oversold have significant scores (weaker side is at least half as strong as stronger side), return **NEUTRAL** — the indicators disagree.
3. **Pick a Winner**: Compare weighted scores. Higher score wins. If tied, return **NEUTRAL**.

4. **Threshold Check:** The winner's weighted score must be **strictly greater than the neutral count.** If not, return **NEUTRAL** — signal too weak against indifferent indicators.

5. **Assign Strength:** If Strong signals outnumber (or equal) Regular signals on the winning side, label it **STRONG_**; otherwise regular.

```
# ================================================================================
# INDIVIDUAL INDICATOR CLASSIFICATION
# ================================================================================

def classify_indicator(value, indicator_name):
    """
    Classify individual indicator into 6 consensus categories.
    """
    if pd.isna(value):
        return 'NEUTRAL'

    THRESHOLDS = {
        # Format: (strong_oversold, oversold, overbought, strong_overbought)
        'RSI': (20, 30, 70, 80),
        'STOCH_K': (10, 20, 80, 90),
        'STOCH_D': (10, 20, 80, 90),
        'STOCH_SLOW_K': (10, 20, 80, 90),
        'STOCH_SLOW_D': (10, 20, 80, 90),
        'STOCHRSI': (0.10, 0.20, 0.80, 0.90),  # 0-1 scale, not 0-100
        'WILLIAMS_R': (-90, -80, -20, -10),     # Inverted logic
        'MFI': (10, 20, 80, 90),
        'ULTIMATE_OSC': (20, 30, 70, 80)
    }

    thresholds = THRESHOLDS[indicator_name]
    strong_os, os, ob, strong_ob = thresholds

    # Williams %R is inverted (higher value = more overbought, closer to 0)
    if indicator_name == 'WILLIAMS_R':
        if value <= strong_os:      # <= -90
            return 'STRONG_OVERSOLD'
        elif value <= os:           # <= -80
            return 'OVERSOLD'
        elif value >= strong_ob:    # >= -10
            return 'STRONG_OVERBOUGHT'
        elif value >= ob:           # >= -20
            return 'OVERBOUGHT'
        else:
            return 'NEUTRAL'
    else:
        # Standard 0-100 scale indicators
        if value <= strong_os:
```

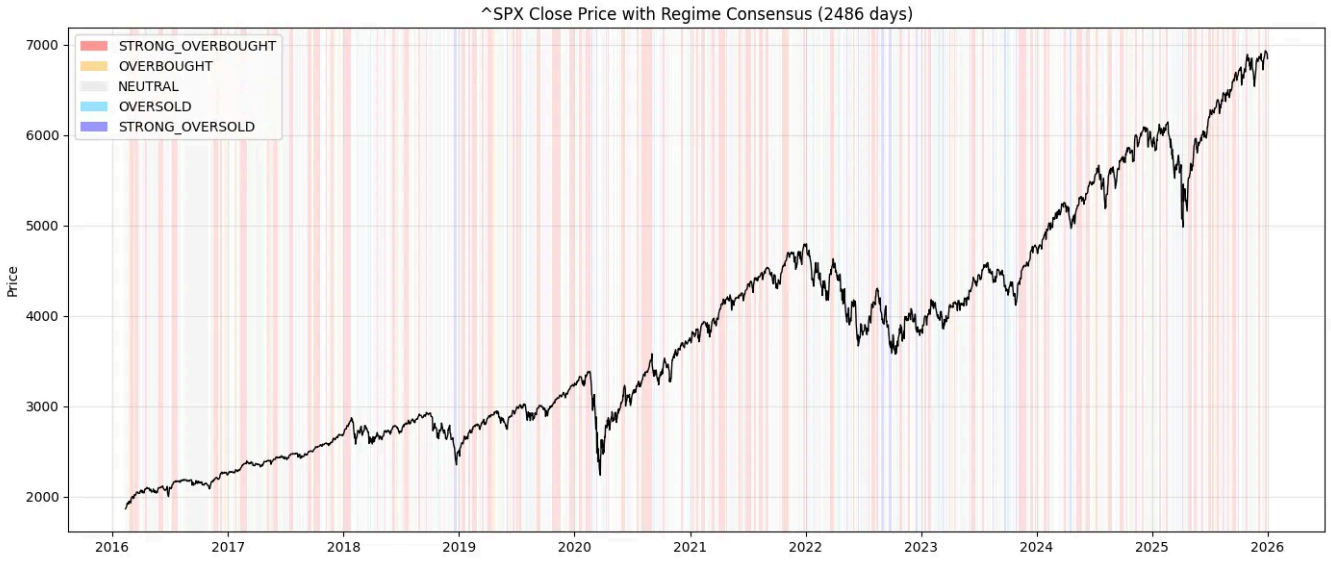
```
            return 'STRONG_OVERSOLD'
        elif value <= os:
            return 'OVERSOLD'
        elif value >= strong_ob:
            return 'STRONG_OVERBOUGHT'
        elif value >= ob:
            return 'OVERBOUGHT'
        else:
            return 'NEUTRAL'
```

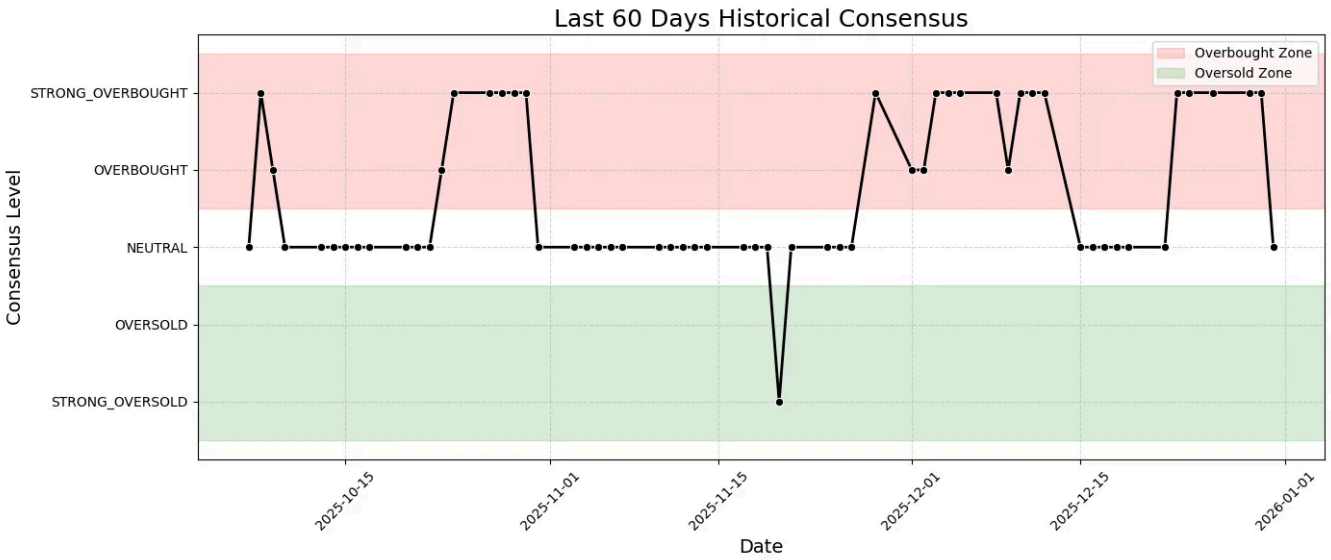
```
# ================================================================================
# CALCULATE CONSENSUS
# ================================================================================

def calculate_consensus(row):
    """
    REFINED CONSENSUS VOTING

    Rules:
    1. Strong signals = 2 points, Regular = 1 point
    2. Winner must have weighted score > neutral_count
    3. If both sides present with similar strength → NEUTRAL (conflict)
    4. Ties or insufficient scores → NEUTRAL
    """

    # Count categories
    strong_ob = sum(1 for x in row if x == 'STRONG_OVERBOUGHT')
    ob = sum(1 for x in row if x == 'OVERBOUGHT')
    strong_os = sum(1 for x in row if x == 'STRONG_OVERSOLD')
    os = sum(1 for x in row if x == 'OVERSOLD')
    neutral = sum(1 for x in row if x == 'NEUTRAL')

    # Weighted scores
    ob_score = strong_ob * 2 + ob
    os_score = strong_os * 2 + os

    # Calculate totals
    total_indicators = len(row)

    # DEBUG INFO
    debug_info = {
        'strong_ob': strong_ob,
        'ob': ob,
        'ob_score': ob_score,
        'strong_os': strong_os,
        'os': os,
        'os_score': os_score,
        'neutral': neutral,
        'total': total_indicators
    }
```

```
    # RULE 1: All neutral
    if ob_score == 0 and os_score == 0:
        return 'NEUTRAL', {**debug_info, 'reason': 'ALL_NEUTRAL'}

    # RULE 2: Conflict detection - both sides present with similar strength
    if ob_score > 0 and os_score > 0:
        max_score = max(ob_score, os_score)
        min_score = min(ob_score, os_score)
        ratio = min_score / max_score if max_score > 0 else 0

        # If weaker side is at least 50% of stronger side → conflict
        if ratio >= 0.5:
            return 'NEUTRAL', {
                **debug_info,
                'reason': 'CONFLICTING_SIGNALS',
                'ratio': round(ratio, 2)
            }

    # RULE 3: Determine winner (higher score)
    if ob_score > os_score:
        winner_score = ob_score
        winner_side = 'OB'
        # Determine strength label
        if strong_ob > ob:
            candidate = 'STRONG_OVERBOUGHT'
        elif strong_ob == ob and strong_ob > 0:
            candidate = 'STRONG_OVERBOUGHT'
        else:
            candidate = 'OVERBOUGHT'

    elif os_score > ob_score:
        winner_score = os_score
        winner_side = 'OS'
        # Determine strength label
        if strong_os > os:
            candidate = 'STRONG_OVERSOLD'
        elif strong_os == os and strong_os > 0:
            candidate = 'STRONG_OVERSOLD'
        else:
            candidate = 'OVERSOLD'

    else:
        # Exact tie
        return 'NEUTRAL', {**debug_info, 'reason': 'TIED_SCORES'}

    # RULE 4: THRESHOLD CHECK - winner must beat neutral
    if winner_score <= neutral:
        return 'NEUTRAL', {
            **debug_info,
            'reason': 'INSUFFICIENT_VS_NEUTRAL',
            'winner_side': winner_side,
            'winner_score': winner_score,
```

```
            'threshold_needed': neutral + 1
        }

    # PASSED ALL CHECKS
    return candidate, {
        **debug_info,
        'winner_side': winner_side,
        'winner_score': winner_score,
        'margin_vs_neutral': winner_score - neutral,
        'margin_vs_opposite': abs(ob_score - os_score)
    }
```

```
================================================================================
CONSENSUS ANALYSIS
================================================================================

Period: 2016-02-12 to 2025-12-31
Data set size: (2486, 32)

--------------------------------------------------------------------------------
DISTRIBUTION (All 5 Outcomes)
--------------------------------------------------------------------------------
  STRONG_OVERBOUGHT    :  828 (33.31%)
  OVERBOUGHT           :  217 ( 8.73%)
  NEUTRAL              : 1274 (51.25%)
  OVERSOLD             :   60 ( 2.41%)
  STRONG_OVERSOLD      :  107 ( 4.30%)
```





Step #3

```
# add 5 Unbounded indicators (features engineering)
#
#  Moving Average Convergence Divergence (MACD)
#  Bollinger Bands
#  Average Directional Index (ADX)
#  Average True Range (ATR)
#  On-Balance Volume (OBV)
#

macd_ind = MACD(close=df2['Close'], window_slow=26, window_fast=12, window_sign=9)
df2["MACD"] = macd_ind.macd()
df2["MACD_SIGNAL"] = macd_ind.macd_signal()
df2["MACD_HIST"] = macd_ind.macd_diff()

bb = BollingerBands(close=df2['Close'], window=20, window_dev=2)
df2["BB_MAVG"] = bb.bollinger_mavg()
df2["BB_HBAND"] = bb.bollinger_hband()
df2["BB_LBAND"] = bb.bollinger_lband()
df2["BB_PBAND"] = bb.bollinger_pband()
df2["BB_WBAND"] = bb.bollinger_wband()

adx_ind = ADXIndicator(high=df2['High'], low=df2['Low'], close=df2['Close'], window
df2["ADX"] = adx_ind.adx()
df2["ADX_POS"] = adx_ind.adx_pos()
df2["ADX_NEG"] = adx_ind.adx_neg()

atr_ind = AverageTrueRange(high=df2['High'], low=df2['Low'], close=df2['Close'], wi
df2["ATR"] = atr_ind.average_true_range()

obv_ind = OnBalanceVolumeIndicator(close=df2['Close'], volume=df2['Volume'])
df2["OBV"] = obv_ind.on_balance_volume()
```

```
df2.dropna(inplace=True)
```

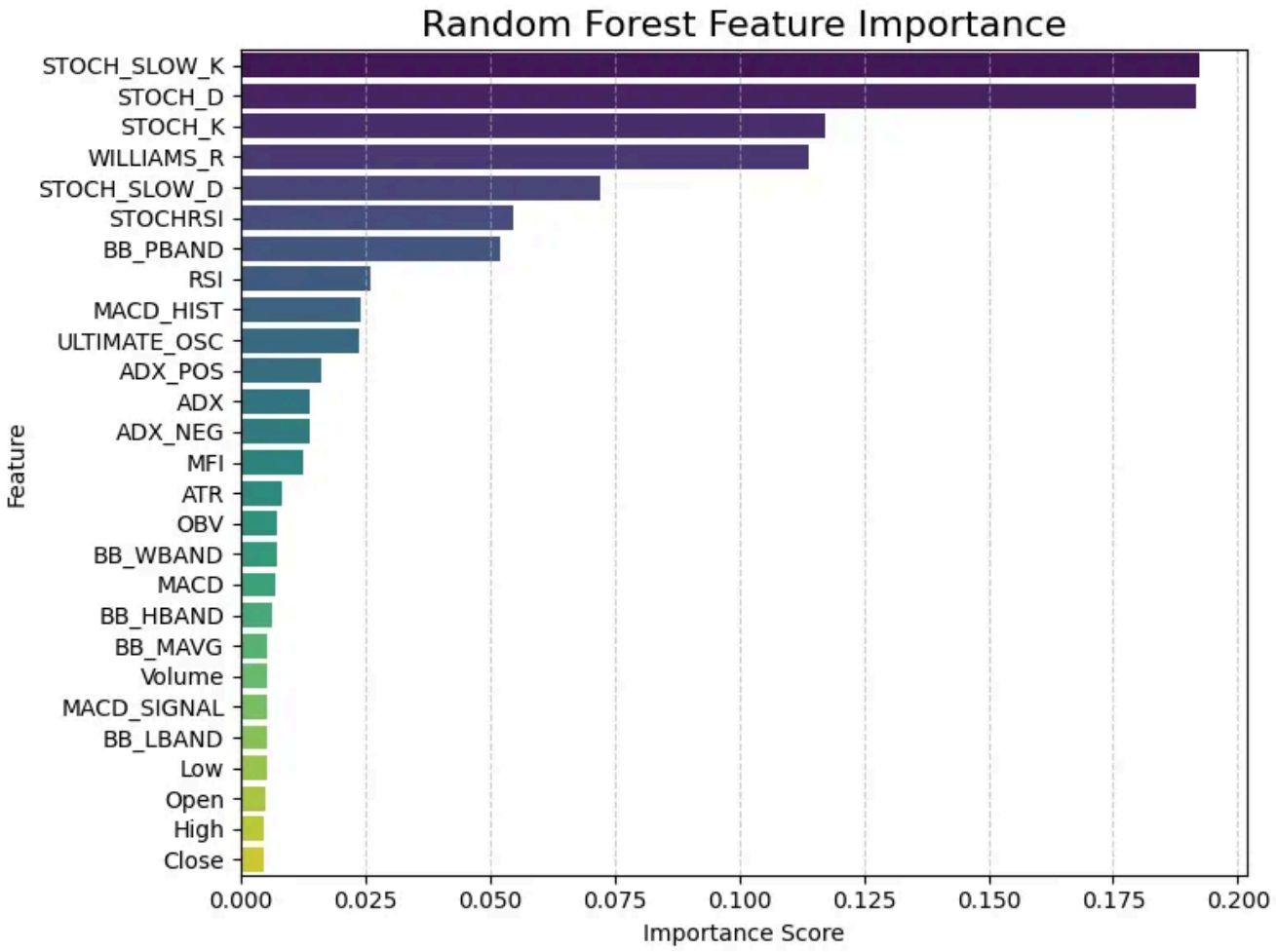
> ***Moving average convergence/divergence (MACD)*** *is a technical indicator used to help investors identify entry points for buying or selling. [7]*
>
> ***Bollinger Bands*** *are a technical analysis tool that shows the volatility of an asset and potential overbought or oversold conditions by plotting two standard deviations away from a simple moving average. [8]*
>
> *The* ***Average Directional Index (ADX)*** *is a useful tool used by traders to measure the current trend of any trading vehicle such as stocks. It does not reveal whether a stock has an bearish or bullish trend. It simply quantifies the strength of the trend. [9]*
>
> ***Average True Range (ATR)*** *is a cornerstone indicator for measuring market volatility. A key characteristic of ATR is that it is not directional; it quantifies the degree of price movement or "choppiness" but does not indicate the trend's direction. [10]*
>
> *Volume Drives Price Action:* ***On-Balance Volume (OBV)*** *is a momentum indicator that uses trading volume to anticipate changes in stock prices. It signals potential price movements based on volume shifts, as volume typically precedes price action. [11]*

Step #4

```
# ----------------------------
#  RF feature selection
# ----------------------------
rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    n_jobs=-1,
    class_weight="balanced"
)
rf.fit(X_train_res, y_train_res)

selector = SelectFromModel(rf, prefit=True, threshold="median")
X_train_sel = selector.transform(X_train_res)
X_test_sel = selector.transform(X_test)

selected_mask = selector.get_support()
selected_features = [f for f, m in zip(feature_cols, selected_mask) if m]
```

```
print("\nSelected features by RF:")
print(selected_features)
```

```
Selected features by RF:
['ADX', 'ADX_NEG', 'ADX_POS', 'BB_PBAND', 'MACD_HIST', 'MFI', 'RSI', 'STOCHRSI', 'S
```



Step #5

```
print("Class distribution BEFORE SMOTE:")
unique, counts = np.unique(y_train, return_counts=True)
for u, c in zip(unique, counts):
    print(f"{idx_to_class[u]}: {c}")

# ------------------------------
# SMOTE + class weights
# ------------------------------
assert df3["TARGET_IDX"].isna().sum() == 0, "NaN labels detected!"
```

```
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

assert set(np.unique(y_train_res)) == {0,1,2,3,4}, "Invalid labels after SMOTE!"


print("\nClass distribution AFTER SMOTE:")
unique_res, counts_res = np.unique(y_train_res, return_counts=True)
for u, c in zip(unique_res, counts_res):
    print(f"{idx_to_class[u]}: {c}")
```

```
Class distribution BEFORE SMOTE:
STRONG_OVERSOLD: 93
OVERSOLD: 52
NEUTRAL: 1024
OVERBOUGHT: 162
STRONG_OVERBOUGHT: 631

Class distribution AFTER SMOTE:
STRONG_OVERSOLD: 1024
OVERSOLD: 1024
NEUTRAL: 1024
OVERBOUGHT: 1024
STRONG_OVERBOUGHT: 1024
```

Step #6

```
# ------------------------------
# CNN model: CNN expects input shaped like: (batch, channels=1, features)
# ------------------------------
import torch
import torch.nn as nn

class DeeperCNN1D(nn.Module):
    """
    12-Layer Deep 1D CNN for Financial Regime Classification
    Balanced depth, controlled channel growth, strong regularization.
    """

    def __init__(self, n_features, n_classes):
        super().__init__()

        def conv_block(in_c, out_c, dropout=0.0):
            return nn.Sequential(
                nn.Conv1d(in_c, out_c, kernel_size=3, padding=1),
                nn.BatchNorm1d(out_c),
```

```
                nn.ReLU(),
                nn.Dropout(dropout)
            )

        # 12 convolutional layers grouped into 6 blocks
        self.conv = nn.Sequential(
            conv_block(1, 32),
            conv_block(32, 32),

            conv_block(32, 64),
            conv_block(64, 64, dropout=0.1),

            conv_block(64, 64),
            conv_block(64, 64),

            conv_block(64, 128),
            conv_block(128, 128, dropout=0.2),

            conv_block(128, 128),
            conv_block(128, 128),

            conv_block(128, 128),
            conv_block(128, 128, dropout=0.3),
        )

        self.global_pool = nn.AdaptiveAvgPool1d(1)

        self.fc = nn.Sequential(
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Dropout(0.4),
            nn.Linear(128, n_classes)
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.global_pool(x).squeeze(-1)
        x = self.fc(x)
        return x

# ------------------------------
# Early stopping
# ------------------------------
class EarlyStopping:
    def __init__(self, patience=10, min_delta=1e-4):
        self.patience = patience
        self.min_delta = min_delta
        self.best_loss = np.inf
        self.counter = 0
        self.early_stop = False

    def __call__(self, val_loss):
        if val_loss < self.best_loss - self.min_delta:
```

```
            self.best_loss = val_loss
            self.counter = 0
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
```

Step #7

```
# ------------------------------
# Scaling
# ------------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_sel)
X_test_scaled = scaler.transform(X_test_sel)

# CNN input: (batch, 1, seq_len)
X_train_cnn = X_train_scaled[:, np.newaxis, :]
X_test_cnn = X_test_scaled[:, np.newaxis, :]

# It converts NumPy arrays → PyTorch tensors
train_dataset = TimeSeriesDataset(X_train_cnn, y_train_res)
test_dataset = TimeSeriesDataset(X_test_cnn, y_test)

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# ------------------------------
#  Model, optimizer, scheduler
# ------------------------------

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


n_features_sel = X_train_cnn.shape[2]
n_classes = len(target_classes)


model = DeeperCNN1D(n_features=n_features_sel, n_classes=n_classes).to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights_tensor.to(device), label_smoot
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)  # Add
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.3, patience=2, verbose=True
)

early_stopping = EarlyStopping(patience=15, min_delta=1e-4)

history = {
```

```
    "train_loss": [],
    "train_acc": [],
    "val_loss": [],
    "val_acc": [],
    "lr": []
}

# ----------------------------
#  Training loop
# ----------------------------
max_epochs = 100
print(f"{'Epoch':<6} {'TrainLoss':<12} {'TrainAcc':<10} {'ValLoss':<10} {'ValAcc':<

for epoch in range(1, max_epochs + 1):
    train_loss, train_acc = train_epoch(model, train_loader, optimizer, criterion,
    val_loss, val_acc, _, _ = eval_epoch(model, val_loader, criterion, device)

    scheduler.step(val_loss)
    lr = optimizer.param_groups[0]['lr']

    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)
    history["lr"].append(lr)

    print(f"{epoch:<6} {train_loss:<12.6f} {train_acc:<10.4f} "
          f"{val_loss:<10.6f} {val_acc:<10.4f} {lr:.6f}")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_epoch = epoch
        torch.save({
            "model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict(),
            "epoch": epoch,
            "val_acc": val_acc
        }, ckpt_path)

    early_stopping(val_loss)
    if early_stopping.early_stop:
        print("Early stopping triggered.")
        break

print(f"\nBest Validation Accuracy: {best_val_acc:.4f} at epoch {best_epoch}")
```

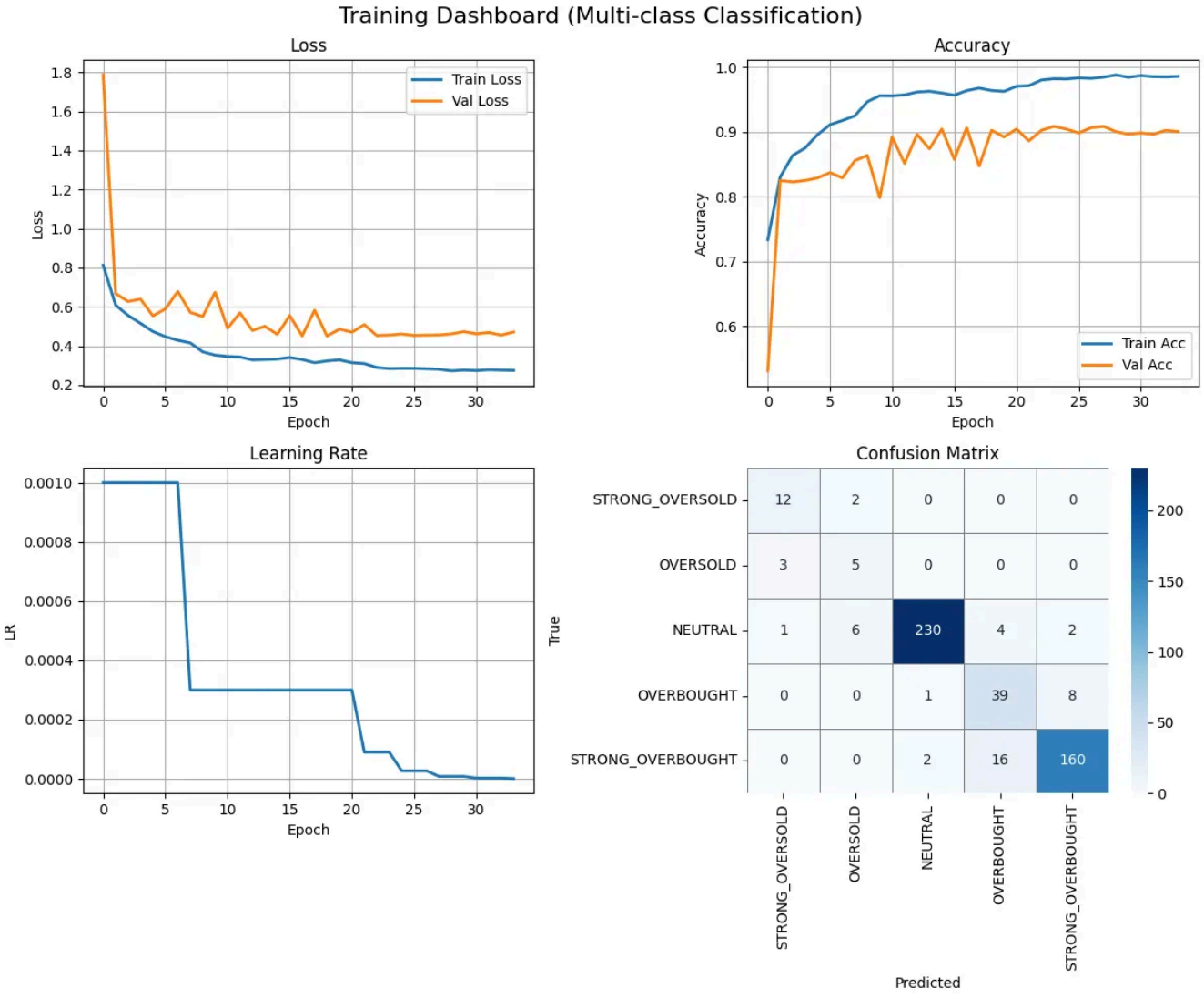
```
Epoch  TrainLoss    TrainAcc   ValLoss    ValAcc     LR
1      0.813179     0.7336     1.786753   0.5316     0.001000
2      0.608779     0.8303     0.667714   0.8248     0.001000
```

```
3      0.556221     0.8635     0.627521   0.8228     0.001000
4      0.515464     0.8750     0.639687   0.8248     0.001000
5      0.473882     0.8955     0.553736   0.8289     0.001000
6      0.447458     0.9109     0.589123   0.8371     0.001000
7      0.428934     0.9174     0.678699   0.8289     0.001000
8      0.415614     0.9246     0.572050   0.8554     0.000300
9      0.369972     0.9463     0.550304   0.8635     0.000300
10     0.352702     0.9557     0.674609   0.7984     0.000300
11     0.346263     0.9555     0.490943   0.8921     0.000300
12     0.343335     0.9568     0.569382   0.8513     0.000300
13     0.328598     0.9613     0.479197   0.8961     0.000300
14     0.330446     0.9625     0.500461   0.8737     0.000300
15     0.333099     0.9598     0.459360   0.9043     0.000300
16     0.340719     0.9564     0.555075   0.8574     0.000300
17     0.330431     0.9635     0.451478   0.9063     0.000300
18     0.314171     0.9674     0.582734   0.8473     0.000300
19     0.323377     0.9637     0.450430   0.9022     0.000300
20     0.328709     0.9623     0.486136   0.8921     0.000300
21     0.314189     0.9701     0.470570   0.9043     0.000300
22     0.309722     0.9711     0.509211   0.8859     0.000090
23     0.289837     0.9799     0.453049   0.9022     0.000090
24     0.283999     0.9818     0.455539   0.9084     0.000090
25     0.285299     0.9814     0.461015   0.9043     0.000027
26     0.285122     0.9832     0.453462   0.8982     0.000027
27     0.283182     0.9824     0.455262   0.9063     0.000027
28     0.280764     0.9842     0.456458   0.9084     0.000008
29     0.272527     0.9877     0.461387   0.9002     0.000008
30     0.276418     0.9840     0.473175   0.8961     0.000008
31     0.273753     0.9867     0.461969   0.8982     0.000002
32     0.278170     0.9850     0.468780   0.8961     0.000002
33     0.276189     0.9846     0.455314   0.9022     0.000002
34     0.274798     0.9855     0.471100   0.9002     0.000001
Early stopping triggered.

Best Validation Accuracy: 0.9084 at epoch 24
```

Step #8

```
Classification Report:
                   precision    recall  f1-score   support

  STRONG_OVERSOLD       0.75      0.86      0.80        14
        OVERSOLD        0.38      0.62      0.48         8
         NEUTRAL        0.99      0.95      0.97       243
      OVERBOUGHT        0.66      0.81      0.73        48
STRONG_OVERBOUGHT       0.94      0.90      0.92       178

        accuracy                            0.91       491
```

```
           macro avg       0.74      0.83      0.78       491
        weighted avg       0.92      0.91      0.91       491

    Confusion Matrix:
    [[ 12   2   0   0   0]
     [  3   5   0   0   0]
     [  1   6 230   4   2]
     [  0   0   1  39   8]
     [  0   0   2  16 160]]
```



Future Improvements

- **Increase model accuracy:** There is still room to further improve predictive performance through feature engineering, model tuning, and better data preprocessing.
- **Expand technical indicators:** Incorporate additional bounded and unbounded technical indicators to enrich the feature set and capture more nuanced market behavior.

- **Refine regime-identification logic:** Adjust the counting and weighting rules used to determine regime labels to improve signal quality and reduce noise.
- **Enhance model architecture:** Explore more advanced CNN architectures or consider hybrid and ensemble approaches, combined with systematic hyperparameter optimization.

Reference:

1. https://commodity.com/technical-analysis/relative-strength-index/
2. https://commodity.com/technical-analysis/stochastics/
3. https://commodity.com/technical-analysis/stochastic-rsi/
4. https://commodity.com/technical-analysis/williams-r/
5. https://commodity.com/technical-analysis/money-flow-index/
6. https://commodity.com/technical-analysis/ultimate-oscillator/
7. https://www.investopedia.com/terms/m/macd.asp
8. https://www.investopedia.com/terms/b/bollingerbands.asp
9. https://medium.com/@arangana/understanding-the-average-directional-index-adx-to-build-trading-strategies-90726498f191
10 https://medium.com/@pyquantlab/technical-indicators-average-true-range-atr-cb35c7ef695d
11 https://www.investopedia.com/terms/o/onbalancevolume.asp

Written by Simon Leung

229 followers · 4 following

A fervent enthusiast, fueled by an insatiable curiosity for STEM (Science, Technology, Engineering, and Mathematics) disciplines.